SAT Resetmaxxing for Fun and Profit
(Final Title TBD)

Martin S. Fixman[1], Fredrik Rømming[2], and Sean B. Holden[3]

University of Cambridge,
United Kingdom
[1]msf43@cl.cam.ac.uk [2]fr409@cl.cam.ac.uk [3]sbh11@cl.cam.ac.uk

Abstract

We improve PySATCoP performance by resetting the connection tableau often. This causes more and more varied SAT statements to be added to the SAT prover, which causes it to saturate faster. The resulting prover solves problems faster and in fewer steps than PySATCoP and the current state-of-the-art tableau provers.

Contents

1 Introduction **1**

2 Preliminaries **1**
2.1 Tableau Solving Methods . . . 1
2.2 SATCoP and Similar Methods . . . 2

3 Research Vehicle **2**
3.1 SATResetCoP Specification . . . 2
3.2 Other things I should add . . . 3
3.3 Python Implementation . . . 4
3.4 TPTP Benchmarking . . . 4

4 Experimental Evaluation **4**
4.1 Experiment Design . . . 4
4.2 Results . . . 5
4.2.1 Individual Results . . . 5
4.2.2 Comparative Results . . . 5
4.2.3 Soundness Verification: Problem Satisfiability . . . 6

5 Future Work **9**

6 Acknowledgements **9**

1 Introduction

[Write an introduction that enhances and complements the abstract.]

2 Preliminaries

2.1 Tableau Solving Methods

Minimalist tableau-style provers based on Prolog have been used with great success since the release of leanTAP in 1995[1], and have consistently achieved high rankings in solving FOF and CNF problems in the Thousands of Problems for Theorem Provers (TPTP) library[16].

Rather than working like saturation provers such as Vampire[12], which repeatedly derive new clauses from the current clause set until they derive a contradiction or reach saturation,

tableau-based provers attempt to create a Connection Tableau with ground terms. The prover builds a tree, where each node corresponds to a particular clause of the problem in conjunctive normal form, using one of the following four steps.

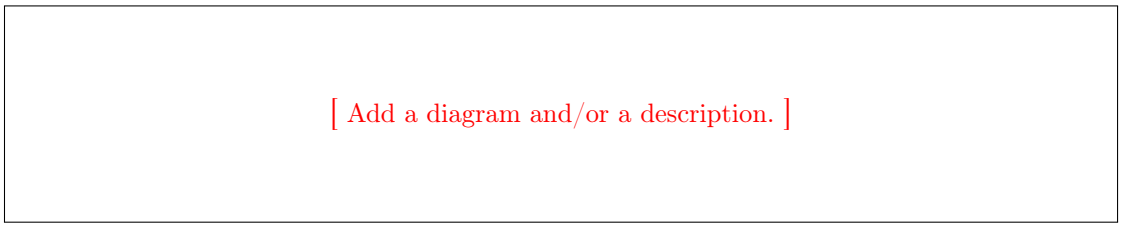


Figure 1: Description of the operations of Tableau calculus[3].

leanCoP improves on earlier connection-tableau provers by combining a very small Prolog implementation with aggressive restricted backtracking[8, 6]. Successful Reduction and Extension steps commit with a Prolog cut (!), pruning alternative continuations and often reducing search substantially. To maximise the variety of substitutions in the tableau the maximum height of the tree is constrained, and each subsequent Start increases this height. A similar technique is later used in SATResetCoP, which is presented in this paper.

meanCoP revisits this design by separating the effects of Reduction cuts and Extension cuts, and by replacing leanCoP's particularly strong pruning with more selective exclusive cuts[3]. In particular, meanCoP commits only when a successful proof step rules out sibling alternatives at the current choice point, and otherwise allows ordinary backtracking to continue[3]. This keeps much of the practical benefit of cut-based control while avoiding some of the unnecessary incompleteness and over-commitment of the original leanCoP strategy[3].

The leanCoP line has also been extended in several other directions. Functional and machine-learning-guided connection provers refine proof-step ordering and clause selection[4], rlCoP and plCoP add Monte Carlo Tree Search and reinforcement-learning-based control to leanCoP-style proof search[5, 18], and recent work on proof syntax and certification makes connection proofs easier to exchange and verify[9].

2.2 SATCoP and Similar Methods

Rawson and Reger integrate a SAT prover into a clausal tableau system based on leanCoP[11]. Grounded clauses generated in Tableau extensions are added to a SAT solver. The SAT solver resulting in an unsatisfiable set of ground clauses is sufficient to prove that the entire problem is unsatisfiable. For the remaining tableau variables, their implementation follows Schulz's suggestion of grounding with the most frequent constant in the conjecture, falling back to a fresh constant if the conjecture contains no constants[11, § 4.2].
[Fill more information here.]

3 Research Vehicle

3.1 SATResetCoP Specification

We propose implementing a new tableau prover, based on SATCoP, but whose tableau prover attempts to saturate the SAT prover rather than to close the tableau.

Successive experimentation resulted in a simple hypothesis: aggressively resetting the tree when the SAT solver stops growing results in a larger quantity and variety of SAT statements added to the solver, which results in reaching a SAT unsatisfiability proof faster.

SATResetCoP uses the same substitution and SAT prover unsatisfiability technique as PySATCoP. However, instead of following leanCoP's tableau guidance strategy, it prioritises resetting the tableau often. The policy is sketched below; the ordering of forward actions follows the SAT-model literal selection idea of Rawson and Reger[11, § 9.1].
[Check this.]

```
def solve_tableau(start_clauses):
  clauses := ground(start_clauses)

  max_depth := 1
  tableau := empty_tableau(max_depth)

  while solve_sat(clauses) != UNSAT:
    Actions := legal_actions(tableau)  // may include Backtrack

    model_rank(lit) :=
      0 if ground(lit) is true in SAT(clauses)
      1 if ground(lit) is unassigned in SAT(clauses)
      2 if ground(lit) is false in SAT(clauses)
      3 if lit = Backtrack

    a := argmin(a in Actions, model_rank(selected_literal_or_backtrack(a)))

    if a != Backtrack:
      step(a)
      clauses.add(ground(tableau))
      continue

    if ground(tableau) adds a new clause:
      tableau := empty_tableau(max_depth)
    else:
      tableau := empty_tableau(max_depth + 1)
      max_depth := max_depth + 1
```

Listing 1: Sketch of the SATResetCoP control policy. Ground clauses are retained across resets; only the tableau is rebuilt.

Just like PySATCoP, this approach is sound. While it is less complete than PySATCoP and other leanCoP variants, the speed at which it proves unsatisfiability of first-order problems can result in a larger amount of problems solved in a bounded amount of time or steps. It might be useful to incorporate SATResetCoP into the initial steps of a more complete schedule.

3.2 Other things I should add

The grounded clauses used by PySATCoP and SATResetCoP come from the clauses currently present in the tableau, after applying the tableau substitution accumulated so far. Ground terms are kept as they are; only tableau variables that remain unassigned need to be replaced by a ground term.

As explained in Section 2.2, Rawson and Reger follow the technique by Schulz[14] to use the most frequent constant in the conjecture, with a fresh constant as a fallback when the conjecture contains no constants[11]. SATResetCoP always uses this fallback-style choice, mapping every remaining tableau variable to the same fresh artificial constant. This gives a grounding scheme

that is simpler than SATCoP, while keeping the proof sound: the SAT solver still only receives ground instances of clauses already present in the tableau.
[Explain how I choose statements.] [Randomisation of statements.]

3.3 Python Implementation

As a testbed for out experiments we use the Connections Prober by Rømming et. al[13], which provides an implementation of leanCoP-style provers.

The connections prover makes it easy to implement different strategies to guide the tableau proof. This makes implementing the guidance of SATResetCoP simple and its code easy to read and evaluate.

The source code for this implementation is present in `https://github.com/mfixman/resetsatcop`, and is released under the GNU Lesser General Public Licence[2].

3.4 TPTP Benchmarking

Along with out implementation of SATResetCoP, PyCoP allows us to reimplement PyLeanCoP and PySATCoP in its own backend. This provides a fairer baseline for comparison.

We run SATResetCoP along with our versions of these two connection tableau strategies, along with a strategy that chooses the first possible action as control, on the 8344 CNF problems and 9091 FOF problems of all 50 categories [Categories?] containing at least one of these problems from the Thousands of Problems for Theorem Provers (TPTP) library[15].

The library provides a varied set of problems that's a standard way to prove first-order provers. It also provides declared statuses for these problems, which are a useful sanity check for false unsatisfiability results.

4 Experimental Evaluation

4.1 Experiment Design

The connections prover provides with PyCoP, a simple prover that chooses the first legal non-backtrack action at each point and resets the tree with an increased maximum height saturated.

As a second control policy we use our implementation of leanCoP[6], PyLeanCoP. This is a Python version of the leanCoP Prolog implementation, with the two recommended predicates used in the first step of the schedule by Otten[6, § 5.5].

`cut` Choose one choice from each extension and reduction and backtrack afterwards, in a similar way to to Prolog's `!` operator.

`comp(7)` The prover does iterative deepening of height 1 at a time when the tree is saturated until reaching maximum height 7. Afterwards limits on height are restricted.

These kinds of simple policies tend to be very effective in tableau provers[7].

The TPTP library contains 8344 CNF problems and 9091 FOF problems separated in 50 categories[15]. Each problem/model pair is run for a maximum of 120 seconds and 1 million steps, on a single core of the University of Cambridge CSD3 Cluster with [add CPU information] and 3370 MiB RAM per core[17].

Model	Explanation	Cut	UNSAT Proof	SAT Guidance	Reset on Backtrack
PyCoP PyLeanCoP	Simple control policy leanCoP with initial set of predicates.	✓			
PySATCoP$_0$	Based on the initial implementation of SATCoP[11].	✓	✓		
PySATCoP	Improvements on SATCoP[11, § 9.1].	✓	✓	✓	
SATResetCoP	Novel model, specified in Section 3.1.	✓	✓	✓	✓

Table 1: The four control models and the test model used in this experiment.

4.2 Results

4.2.1 Individual Results

SATResetCoP considerably improves the amount of problems solved by both PySATCoP$_0$ and PySATCoP under the 120 second time limit, as shown in Table 2. This is mostly caused by the SAT solved reaching unsatisfiability in fewer time and steps than in previous models, as seen in Figures 2a and 2b.

Model	CNF	FOF	Total
PyCoP	205	587	792
PyLeanCoP	701	1155	1856
PySATCoP$_0$	787	1042	1829
PySATCoP	1027	1396	2423
SATResetCoP	1227	1634	2861

Table 2: Amount of CNF and FOF problems solved within the time constraint by each model.

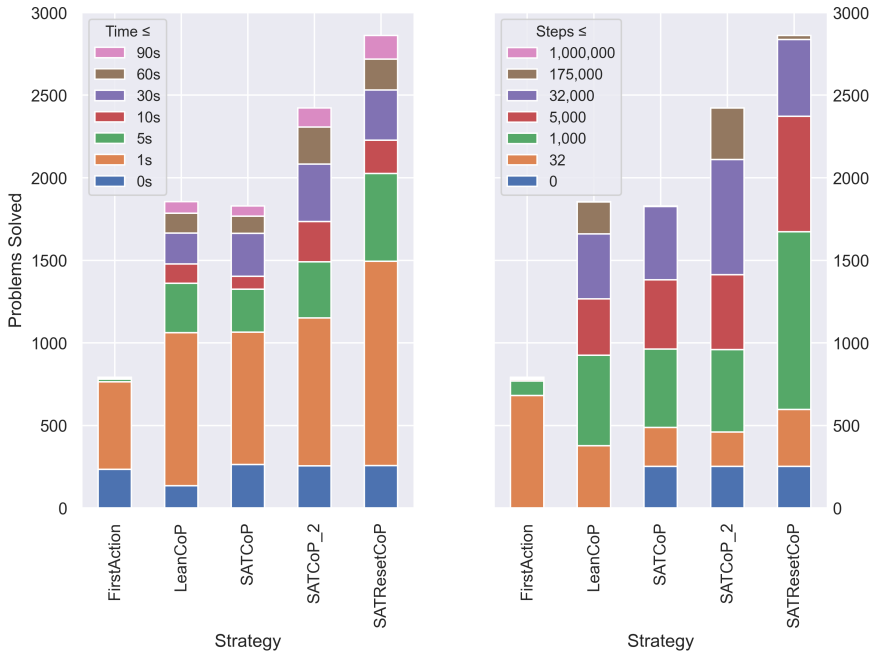
The 1 000 000 step limit is rarely reached, and most unsolved problems hit the 120 second limit instead.
[Maximum solved time is slightly less than 90s. Is this on purpose? Probably; check the actual experiment.]

4.2.2 Comparative Results

The SAT guidance in SATResetCoP causes it to solve several problems that are unsolved by either models which do not have the same SAT saturation prover, and even by PySATCoP$_0$ and PySATCoP, which do not have similar efficient guidance. A comparative result can be seen in Table 3.

SATResetCoP is specially strong in problems which can be solved by small unsat cores. For example, Figure 4 shows the solution to the TPTP problem SYN052-1, which is Pelletier Problem 22[10].

Figure 6 shows the amount of problems solved one of the models that's not solved by another one.



(a) Problems solved, stacked by time in seconds.

(b) Problem solved, stacked by amount of steps. SAT models can solve a problem in 0 steps if the initial grounding produces is SAT-unsatisfiable.

Figure 2: Results of running the models on all problems solved in a 120 second and 1 million step limit.

[Time comparison / violinplot of different TPTP problem types for SATResetCoP]

Figure 3: Time taken by SATResetCoP to solve each problem within each TPTP problem type. We can probably find some conclusion in here.

[There is a lot of interesting data on size of unsat cores and TPTP categories. Investigate further.]

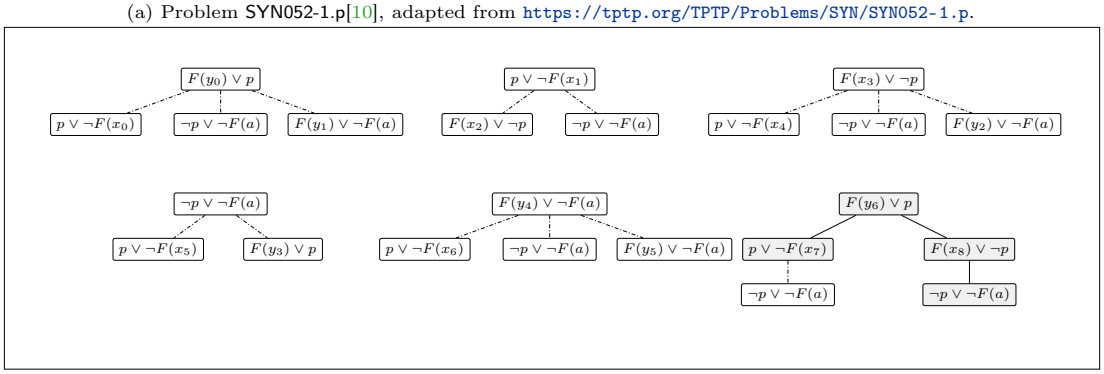
The size of the improvement between SATResetCoP and the next best model is notable. [Add more analysis here.]

4.2.3 Soundness Verification: Problem Satisfiability

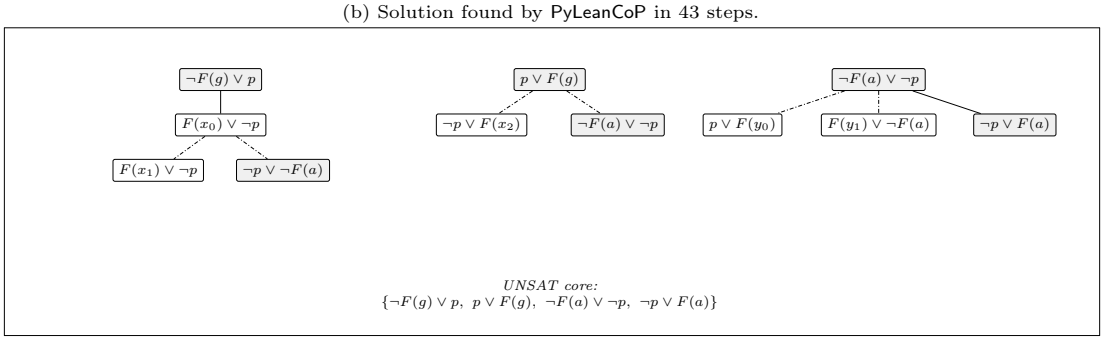
The problems in the TPTP library have a helpful specification of the satisfiability of different types of problems[15]. This is useful as a sanity-check: a sound solver should never "prove" unsatisfiability on a satisfiable problem, and their authors should be wary when proving unsatisfiability on an open or unknown problem. Note that this is not a way to prove soundness

$$p \vee \neg F(X) \qquad F(X) \vee \neg p$$
$$F(Y) \vee p \qquad \neg p \vee \neg F(a) \qquad F(Y) \vee \neg F(a)$$

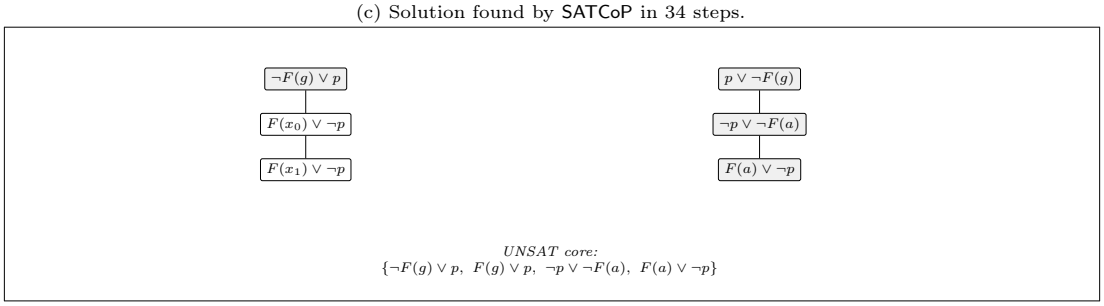
(a) Problem SYN052-1.p[10], adapted from https://tptp.org/TPTP/Problems/SYN/SYN052-1.p.



(b) Solution found by PyLeanCoP in 43 steps.



(c) Solution found by SATCoP in 34 steps.

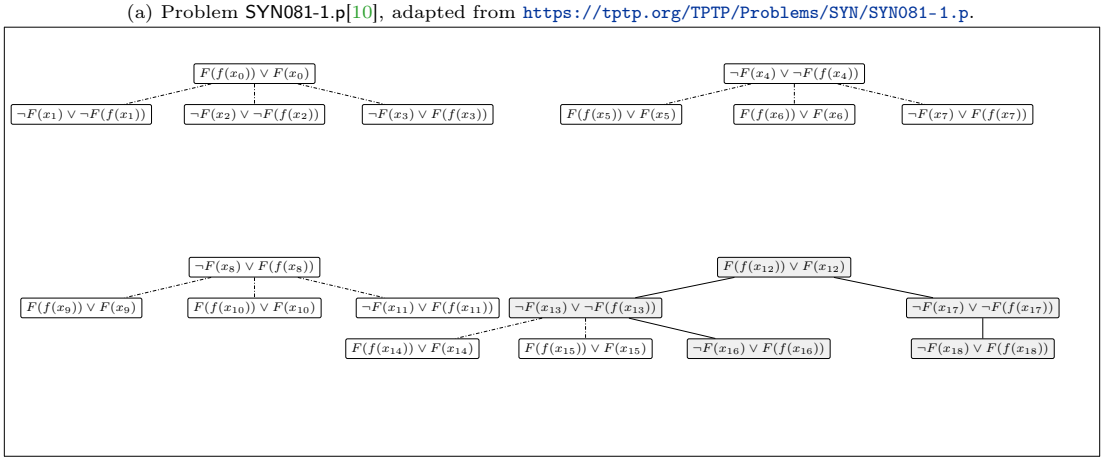


(d) Solution found by SATResetCoP in 11 steps.

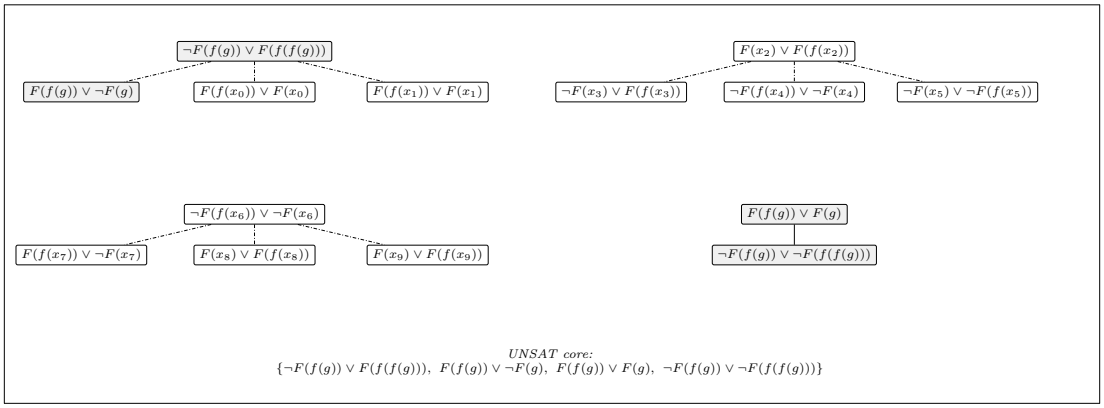
Figure 4: The unsatisfiability proof to the TPTP problem SYN052-1[10] coming from three provers: PyLeanCoP, PySATCoP, and SATResetCoP.

$$\neg F(X) \vee \neg F(f(x)) \qquad F(f(X)) \vee F(x)$$
$$\neg F(X) \vee F(f(X))$$

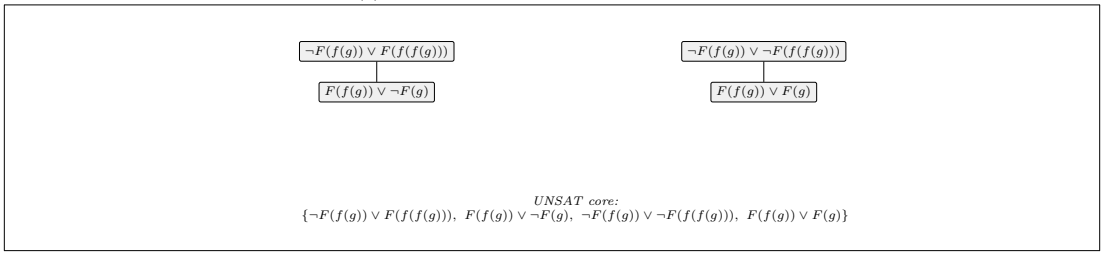
(a) Problem SYN081-1.p[10], adapted from https://tptp.org/TPTP/Problems/SYN/SYN081-1.p.



(b) Solution found by PyLeanCoP in 43 steps.



(c) Solution found by SATCoP in 34 steps.



(d) Solution found by SATResetCoP in 11 steps.

Figure 5: The unsatisfiability proof to the TPTP problem SYN081-1[10] coming from three provers: PyLeanCoP, PySATCoP, and SATResetCoP.

Model	CNF	FOF
PyCoP	205	587
PyLeanCoP	502	693
PySATCoP$_0$	184	129
PySATCoP	188	189
SATResetCoP	213	228

Table 3: Problems solved by each model and not solved by any preceding model in the table. [What's the point of separating CNF and FOF here? Maybe finding interesting TPTP types would be better.]

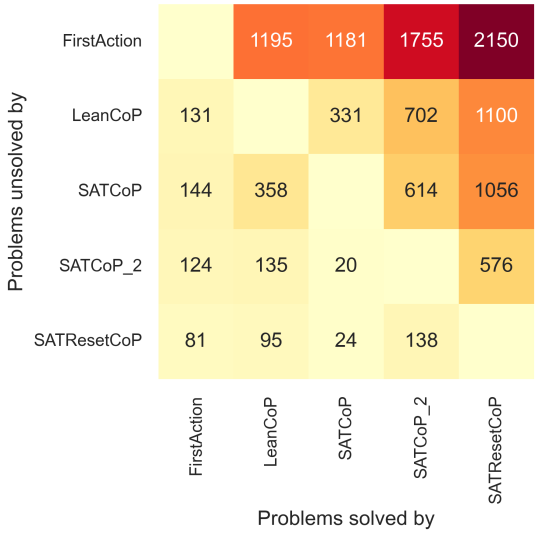


Figure 6: Pairwise comparison of model results: the cell in column y and row x shows the amount of problems solved by the model on y, but not the model in x. For example, SATResetCoP solved 1056 TPTP problems that PySATCoP$_0$ did not. [Remake this figure with new names.]

of the solver, but to check that it does not exhibit false unsatisfiability results on problems declared satisfiable or countersatisfiable.

We make a second run of SATResetCoP with a longer 10-minute limit, whose results can be seen in Table 4. These results do not show any negative results.

5 Future Work

6 Acknowledgements

This work was performed using resources provided by the Cambridge Service for Data Driven Discovery (CSD3) operated by the University of Cambridge Research Computing Service (www.csd3.cam.ac.uk), provided by Dell EMC and Intel using Tier-2 funding from the Engineering and Physical Sciences Research Council (capital grant EP/T022159/1), and DiRAC funding from the Science and Technology Facilities Council (www.dirac.ac.uk).

	FOF		CNF	
Declared status	Unsolved	Solved	Unsolved	Solved
Satisfiable	350	0	1094	0
CounterSatisfiable	732	0	0	0
Unknown	94	0	745	0
Open	4	0	28	0
Unsatisfiable	100	72	4496	1433
ContradictoryAxioms	204	39	0	0
Theorem	4426	1823	0	0

Table 4: Amount of problems solved and unsolved by SATResetCoP, grouped by TPTP declared status. No satisfiable or countersatisfiable problem was incorrectly proven to be unsatisfiable.

References

[1] Bernhard Beckert and Joachim Posegga. leanTAP: Lean tableau-based deduction. 15(3):339–358, 1995.

[2] Free Software Foundation. GNU lesser general public license, version 3, 2007.

[3] Michael Färber. A Curiously Effective Backtracking Strategy for Connection Tableaux, 2024.

[4] Michael Färber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance for Connection Tableaux. 65(2):287–320, 2021.

[5] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Miroslav Olšák. Reinforcement learning of theorem proving. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 8836–8847. Curran Associates, Inc., 2018.

[6] Jens Otten. Restricting backtracking in connection calculi. 23(2–3):159–182, 2010.

[7] Jens Otten. 20 years of leanCoP — an overview of the provers. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*, volume 3613 of *CEUR Workshop Proceedings*, 2023.

[8] Jens Otten and Wolfgang Bibel. leanCoP: Lean connection-based theorem proving. 36(1–2):139–161, 2003.

[9] Jens Otten and Sean Holden. A Syntax for Connection Proofs, 2023.

[10] Francis Jeffry Pelletier. Seventy-five problems for testing automatic theorem provers. 2(2):191–216, 1986.

[11] Michael Rawson and Giles Reger. Eliminating Models During Model Elimination. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 250–265. Springer International Publishing.

[12] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. 15(2–3):91–110, 2002.

[13] Fredrik Rømming, Jens Otten, and Sean B. Holden. Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, volume 3613 of *CEUR Workshop Proceedings*, pages 107–118, 2023.

[14] Stephan Schulz. Light-weight integration of SAT solving into first-order reasoners: First experiments. In Laura Kovács and Andrei Voronkov, editors, *Vampire 2017. Proceedings of the 4th Vampire Workshop*, volume 53 of *EPiC Series in Computing*, pages 9–19. EasyChair, 2018.

[15] Geoff Sutcliffe. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. 59(4):483–502, 2017.

[16] Geoff Sutcliffe. The 12th IJCAR automated theorem proving system competition—CASC-J12. 38(1):3–20, 2025.

[17] University of Cambridge Research Computing Services. Ice Lake Nodes. https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html, 2026. Accessed 13 May 2026.

[18] Zsolt Zombori, Josef Urban, and Chad E. Brown. Prolog Technology Reinforcement Learning Prover: (System Description). In Nicolas Peltier and Viorica Sofronie-Stokkermans, editors, *Automated Reasoning*, volume 12167, pages 489–507. Springer International Publishing, 2020.